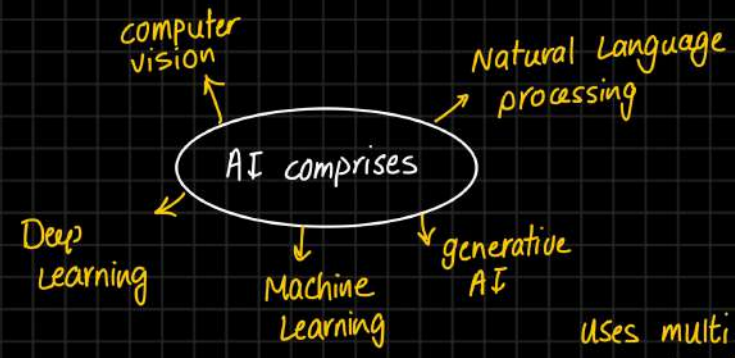
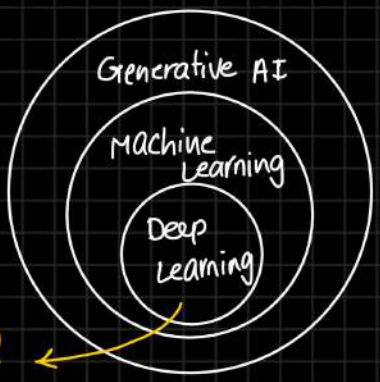
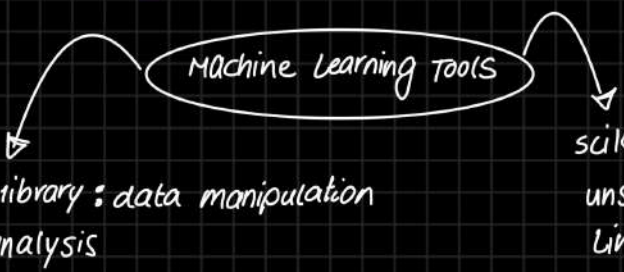




* Machine Learning paradigms:
- supervised learning models train on labeled data
- unsupervised " works without labels, but finding patterns
- semi-supervised learning works iteratively
- reinforcement learning learns from feedback

* ML techniques:
- classification → predicts class or category of a case.
- regression → predicts continuous values
- clustering → groups similar data (in banking field) → unsupervised
- association → find items or events that co-occur
- anomaly detection → discovers abnormal and unusual cases (credit card fraud detection)
- sequence mining → predicts next event
- dimension reduction → reduces size of data
- recommendation systems → associate peoples preferences

clean data → select algorithm → train model

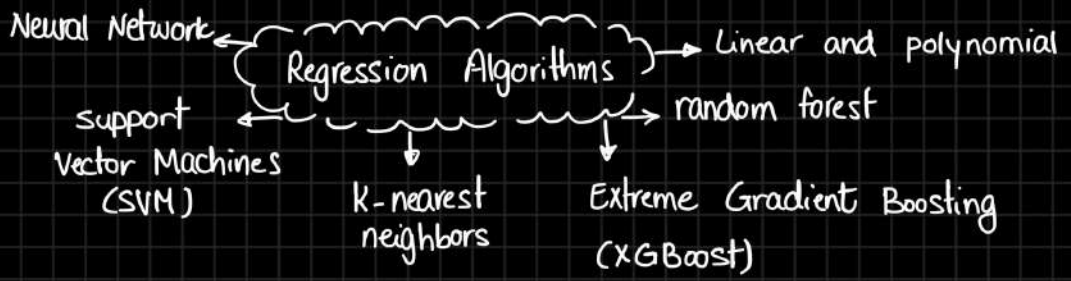


- **Regression** → supervised learning model
  - models a relationship between a continuous target variable and explanatory features
  - **Simple**
    - linear
    - non_linear
  - **Multiple** (when more than one independent variable is present)
    - linear
    - non-linear

→ Applications: sales forecasting, price estimation, predictive maintenance, employment income

↓

rainfall estimation, wildfire probability and severity, spread of infectious disease, risk of chronic disease



Simple Linear Regression:

engine size	cylinders	fuelconsumption_comb	(y) CO2 emission (we want to predict)

- first => plot emisson vs. each of variables

$\hat{y} = \theta_0 + \theta_1 x_1$ (slope; x_1: here: engine size)

$\hat{y}$: emission (predicted); θ_0: intercept

$$\theta_0 = \bar{y} - \theta_1 \bar{x}$$

$$\theta_1 = \frac{\sum (x_i - x)(y_i - y)}{\sum (x_i - x)^2}$$

MAE (Mean Absolute Error): $$MAE = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i|$$

* MSE (Mean Square Error): $$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

RMSE (Root Mean Square Error): $$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2}$$

R^2 sore: $$R^2 : 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum_{i=1} (y_i - \bar{y})^2}$$

• Multiple Linear Regression:

↳ uses two or more independent variables

Regression model: $\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$

$X = [1, x_1, \ldots, x_n]$ feature vectors

$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}$ weights

$\hat{y} = X\theta$ →
- 1 feature $\hat{y} = \theta_0 + \theta_1 x_1$ (line)
- 2 features $\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2$ (plane)
- N features $\hat{y} = \theta_0 + \theta_1 x_1 + \ldots + \theta_n x_n$ (an N-dimensional hyperplane)

be cautious: too many variables can cause overfitting

• Polynomial and non-linear regression:

↳ useful for complex relations

* cubic model example: $\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2 + \cdots$
↳ transform input variable => $x = x_1$, $x^2 = x_2$, $x^3 = x_3$

linearized model: $\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \cdots$

- exponential or compound growth
- logarithmic
- periodicity

• Logistic Regression:

predicts the probability of an observation belonging to one of two classes.
↳ in ML, it refers to a binary classifier based on statistical logistic regression

predicting detector => especially in medicine

probability to class predictions: $\hat{p} = \sigma(\hat{y}) = \frac{1}{1 + e^{-\hat{y}}}$

decision boundary

$\sigma(\hat{y}) \rightarrow \begin{cases} 0 & \text{if } \sigma(\hat{y}) < 0.5 \\ 1 & \text{if } \sigma(\hat{y}) > 0.5 \end{cases}$

• Training a logistic regression:

1. choose a set of parameters or θ
2. predict probability that class = 1
3. calculate prediction error (cost function)
4. update θ to reduce prediction error
5. repeat until → reach small log-loss value or targeted # of iterations.

measures how well $(\hat{p}_i)$ matches y_i

cost function or log-loss needs to be minimized

optimizing logistic regression

$\log\text{-loss} => -\frac{1}{N} \sum_{i=1}^{N} y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)$ →

- * confident and correct => predicted probability of class 1 is ↑ and correct => log-loss is small. ($\hat{p}_i = 1$, $y_i = 1$)
- * confident and incorrect => predicted probability of class 0 is ↑ and incorrect => log-loss is large ($\hat{p}_i \approx 1$, $y_i = 0$)

* Minimizing cost_function with gradient descent:
  - iterative approach to find the minimum of a function.
  - adjust parameter values using Log-loss derivative

slope ↑ longer the step => Best parameters at minimum cost function

Large dataset => slow descent

* Stochastic gradient descent (SGD):
  - uses a random data subset and scales well.
  - likely to overlook local and find global minima.
  - converges quickly toward a global minima.

- Classification:
  - Supervised ML method
  - uses fully trained models to predict labels on new data
  - Labels form a categorical variable with discrete values

↳ Applications => email filtering, speech_to_text, handwriting recognition, biometric identification, document classification

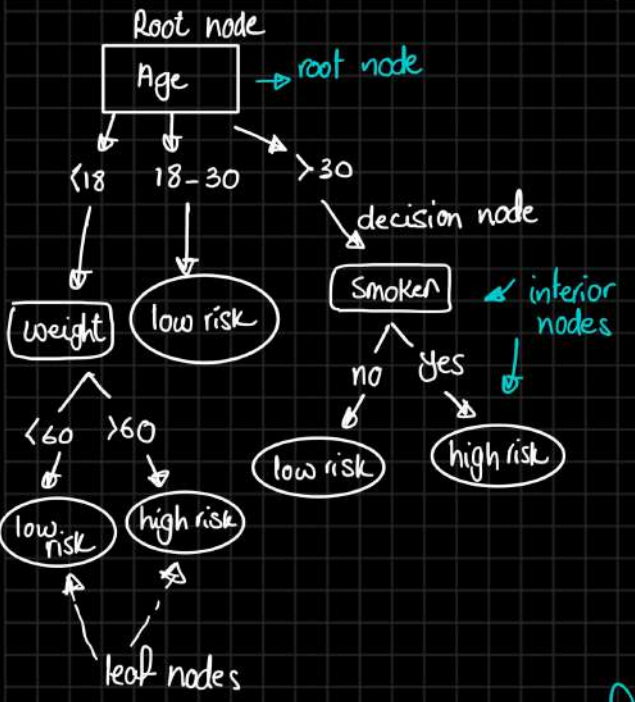
can classify multiple distinct labels

Classification algorithms => Naive Bayes, Logistic Regression, Decision trees, K-nearest neighbors, support vector machines, neural networks

- Multiclass prediction:
  - strategies
    - One-versus-all
      - binary classifier => one for each class label
        - □ → red classifier / ▨ → blue ... classifier
      - * task => binary prediction for every data point for a one-versus-the-rest classifier
      - * K-classes => K binary classifiers
    - one-versus-one
      - red
      - blue
      - green
      - yellow
      - red / blue → red / green / yellow → is it red or is it blue?
      - blue / green → blue / yellow → is it blue or green?
      - voting (popularity)

• Decision Trees: classifier

- each internal node corresponds to a test.
- each branch corresponds to the result of the test.
- each terminal, or leaf node, assigns its data to a class.



* train a decision tree

1. start with a seed node and labeled training data
2. find the feature that best splits the data
3. each split partitions the node's input data.
4. repeat the process for each new node

* Stop growing the tree when:

1. maximum tree depth is reached
2. minimum # of data points in a node has been exceeded.
3. minimum # of samples in a leaf has been exceeded.
4. decision tree has reached maximum # of leaf nodes.

Or prune

- simplifies decision trees
- more concise and easier to understand
- results in better predictive accuracy

common split measures =>
- information gain → entropy reduction
- Gini impurity

* Entropy → measures how random the classes in a node are

1 drug A, 7 drug B → entropy ↓

3 drug A, 5 drug B → entropy ↑

0 drug A, 8 drug B → entropy ↓

4 drug A, 4 drug B → entropy ↑

• Regression Trees:

* in classification => target is categorical

* in regression => target is continuous

	Classification Trees	Regression Trees
target variable	categorical	float
splitting criterion	gini impurity or entropy	variance reduction
prediction at leaf nodes	class label majority vote	average value of target values

example use cases — spam detection, image classification, medical diagnosis — predicting revenue, temperatures, wildfire risk

- Regression trees => recursively split dataset into subsets to maximize information gain.
  - minimizes randomness of classes assigned to split nodes
  - generates a tree-like structure

- creating regression trees:



→ predicted value: Mean of target values

$$\hat{y} = \frac{1}{n} \sum_{i=1}^{n} y_i$$

→ alternative value: . Median of target values
. Better for skewed data
. More expensive

* Splitting criterion: features that minimize error between actual and predicted value

MSE: Mean Squared Error $\frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y})^2$

* Quality of split: (weighted average of MSEs of each split)

$$MSE_{Avg} = \frac{1}{N_{total}} (N_{left} * MSE_{left} + N_{right} * MSE_{right})$$

=> the lower this value the lower the variance => the higher the quality of the split

CHOOSING THE BEST SPLIT

calculate MSE for left and right modules → calculate weighted average of MSE → select split with smallest value

* Categorical feature splits:

Binary feature
- seperate into two classes
- calculate weighted average of MSEs.
- No minimization needed

categorical features (with multiple levels)
- evaluate different binary partitions of the categories and select the split that minimizes the weighted MSE.

* Continuous feature trial thresholds:

Sort feature values: $x_i \leq x_j$ for all $i < j$ → drop duplicates: $x_i < x_j$ for all $i < j$ → define midpoint thresholds: $\alpha_i = \frac{x_i + x_{i+1}}{2}$

choose α that minimizes weighted MSE

for large datasets, select a sparse subset

↳ assumption: target values are uniformly distributed → consider distribution when sampling thresholds

• Supervised Learning with SVMs:

for building classification and regression models

classifies input by identifying hyperplane

the main goal: create a hyperplane that segregates data set.



incorporate soft margin

tolerate misclassifications while maximizing the margin

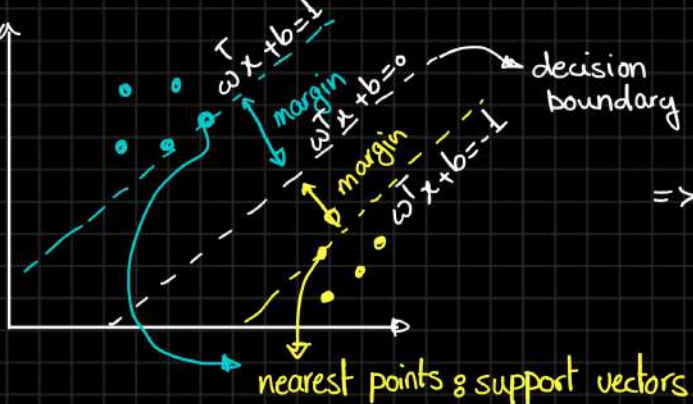
balance between maximizing margin controlled by parameter C

smaller C allows more misclassifications (softer margin)

larger C forces a stricter seperation (harder margin)

SVM => Divide data into two classes by finding decision boundary

a hyperplane that maximizes margin



=> find a weight vector and value b.

(Kernelling)
2D → 3D

* Applications =>
- image classification and handwritten digit recognition
- Parsing, spam detection, sentiment analysis
- speech recognition, anomaly detection, and noise filtering.

- **Supervised Learning with K-NN:** → used for both classification and regression
  - uses labeled points to learn how to label other points
  - neighbors => data points near each other with similar features.

K-NN
- pick a value for K
- for each query point
  - calculate distances to labeled points
  - select K-nearest neighbors
  - predict class
    - classification → most popular class
    - regression → average or median of target values

* finding the optimal K:

1) test a range of K values
2) choose K=1, and calculate accuracy
3) repeat increasing K
4) see which one gave the best accuracy

k-NN is a lazy learner
- memorizes training data
- makes predictions based on distance to training data points.

Brute force algorithm:

for each query point → calculate distances → sort ascending → select top K labels → assign class or value

K too small → values fluctuate overfitting

K too large → finer details lost underfitting

- **Bias, Variance, and ensemble models** → high score: low variance + low bias
  - Bias: indicates how "on target" predictions are.
  - Variance: how spread out darts are

* prediction bias:

Measures the accuracy of predictions: → reflects differences from target values.

Average prediction − average of actuals

$$\text{Bias} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)$$

zero for perfect predictors: $\hat{y}_i = y_i$ for all i

* variance → high variance shows sensitivity to training data.
  ↓
  . leads to overfitting training data
  . tracks noise in training data

low variance => less sensitive to noise

weak learners => { high bias → leads to underfitting
                 { low variance

strong learners => { low bias → overfitting
                   { high variance

Bagging and Boosting : popular methods for ensemble learning
↓
effective at balancing bias and variance

* perform the process multiple times + Average predictions from multiple iterations
↳ Bagging (or Bootstrap aggregating) → reduces prediction variance + lowers the risk of overfitting

Boosting
↓
builds a series of weak learners
↓
each learner corrects the previous learner's error
↓
boosting systematically reduces prediction error
↓
the final model is a weighted sum
↓↓
weights → Increase weights for misclassified data
          Decrease weights for correctly classified data
          Reweighting focuses on correcting mistakes
          Update model weights based on performance

random forests
↓
use bagging for training
↓
train decision trees on bootstrapped datasets
↓
focus on minimizing prediction bias

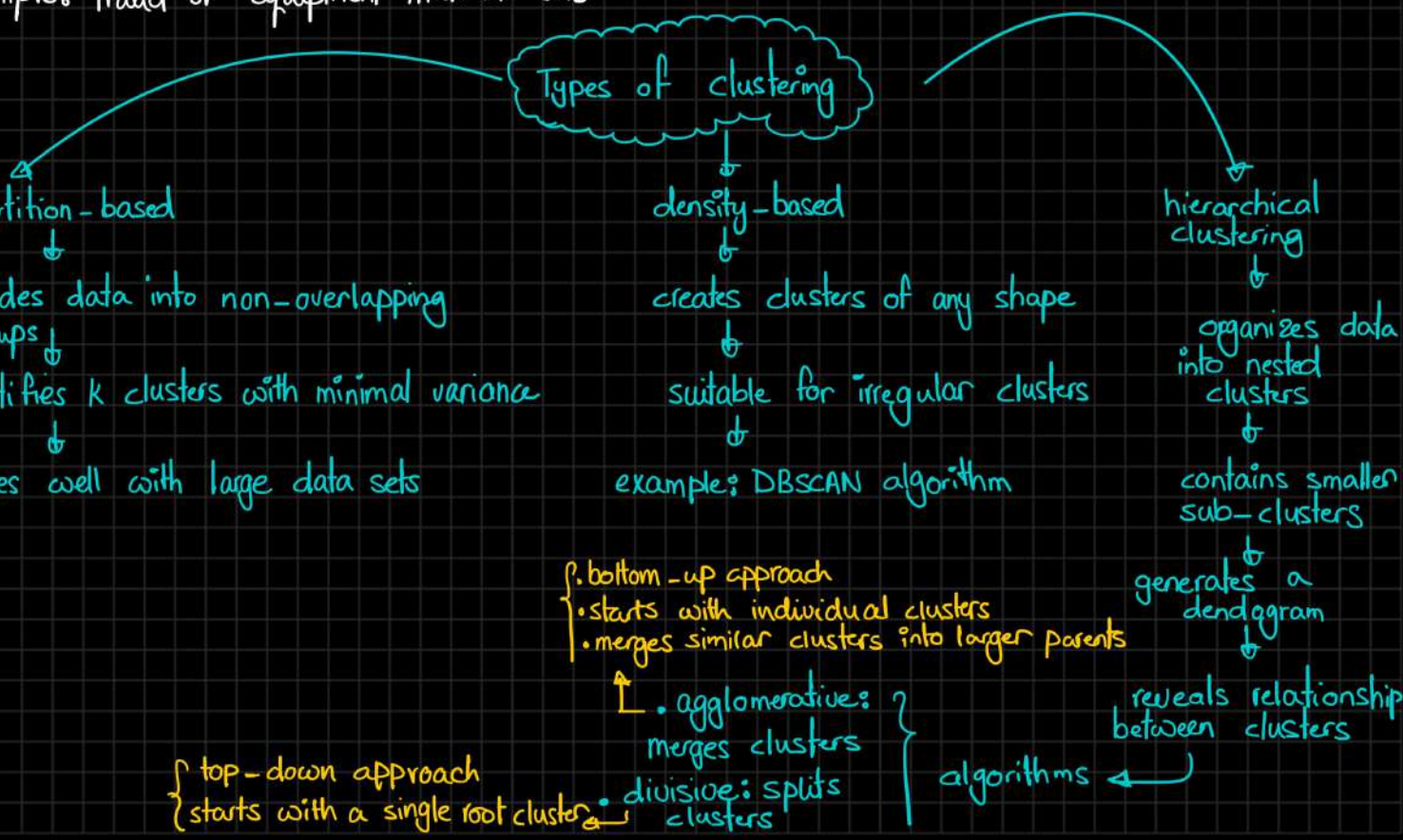
shallow trees
↓
have high prediction variance
↓
focus on minimizing prediction bias

=> popular boosting algorithms { Gradient Boosting
                               { XGBoost
                               { AdaBoost

Ensemble	Objective	Base learners	Training	Outcome
Bagging	Mitigate overfitting	high variance, low bias	parallel on bootstrapped data	reduced variance
Boosting	Mitigate underfitting	low variance, high bias	builds on previous result	reduced bias

- Clustering:
  - automatically groups data points based on similarities
    - identifying music genres / segmenting user groups / analyzing market segments
      - can use 1 or multiple features

- Exploratory data analysis:
  - uncovers natural groupings for targeted marketing
  - example: customer segmentation
- pattern recognition:
  - groups objects and aids in image segmentation
  - example: detecting medical abnormalities
- anomaly detection:
  - identifies outliers
  - example: fraud or equipment malfunctions

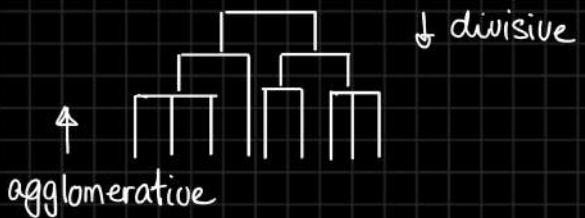


* agglomerative :

creating a distance matrix => update it => closest points ~> create a cluster

↓
distances between each pair of points

$$\text{Matrix} = \begin{bmatrix} 0 & & & \\ d(2,1) & 0 & & \\ d(3,1) & d(3,2) & 0 & \\ \vdots & & & \\ d(n,1) & d(n,2) & \cdots & 0 \end{bmatrix}$$



* Divisive:

Begin with a single cluster => partition this single cluster into smaller clusters
↳ consider the entire dataset as one cluster
↳ based on similarities or disimilarities

=> continue until reaching the minimum # of clusters

• k-means clustering:

↓

Partitions data into similar groups based on distance between centroids

↓

divides data into K non-overlapping clusters

↓

k-clusters have minimal variances around centroids and maximal dissimilarity between clusters

- higher K = smaller clusters with great detail
- lower K = larger clusters with less detail

algorithm:

1) initialize the algorithm:
   - select the # of clusters, k
   - randomly select K centroids
2) Iteratively assign points to clusters and update centroids:
   - compute distance matrix
   - assign each point to cluster with nearest centroid
   - update cluster centroids as the mean position.
3) Repeat until centroids stabilize or max iterations reached.

K-means =>
- assumes convex clusters
- assumes balanced cluster sizes
- sensitive to outliers and noise
- scales well to big data

* k-means optimization:

Goal: Minimize within-cluster sum of squares: $\sum_{i=1}^{k} \sum_{x \in C_i} \| x - \mu_i \|^2$

(x: data point; μ_i: centroid of cluster C_i; i: i-th cluster)

choosing k is feasible when:

data is separable → difficult to visualize for high-dimensional spaces → consider scatterplots between variable pairs to check for separability

determining K:

Heuristic techniques for determining K:

- Silhouette analysis: Measures cohesion and separation
- Elbow method: plot for different cluster numbers
- Davies-Bouldin Index: Measures each clusters average similarity ratio

• DBSCAN and HDBSCAN clustering:

density-based spatial clustering algorithm

↓

creates clusters centered around spatial centroids

↓

user provides density value

↓

defines neighborhoods of clusters with specified density

↓

discovers clusters of any shape, size, or density

↓

distinguishes between data points that are part of a cluster and noise

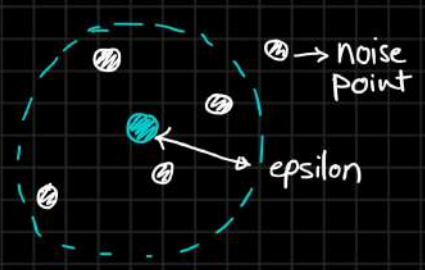
↓

useful for data with outliers or when cluster number is unknown

parameters => N, epsilon

Every data point is labelled as:

- core point: has at least N points within the epsilon neighborhood.
- border point: Non-core points within a core-point neighborhood.
- Noise point: isolated from all core-point neighborhoods



N=5

core point

border point=4

DBSCAN => not iterative

grows clusters in one pass without updating them once they are labeled.

any unassigned points remaining are regarded as noise

* HDBSCAN

↓

cluster stability: cluster persistence over a range of neighborhood sizes

↓

locally adjusts neighborhood radii for cluster stability

↓

improves clustering for variable-density and noisy data

algorithm → combination of agglomerative and density-based clustering → starts by identifying each point as its own cluster → progressively agglomerates clusters into a hierarchy

gets simplified into a condensed tree with stable clusters

• Clustering, dimension reduction, and feature engineering:

complementary techniques in ML and data science

dimension reduction before clustering → used as preprocessing

simplifying data structure and improving outcomes

=> PCA, t-SNE, UMAP

• dimension reduction algorithms:

↓

reduce data set features without sacrificing critical information

- principle component analysis (PCA)
- t-distributed stochastic neighbor embedding (t-SNE)
- uniform manifold approximation and projection (UMAP)

↓

transform original dimensions to create new features

* PCA:

assumes data set features are linearly correlated → simplifies data, reduces dimensionality and reduces noise and minimizes information loss → can transform features into principle components and retain variance

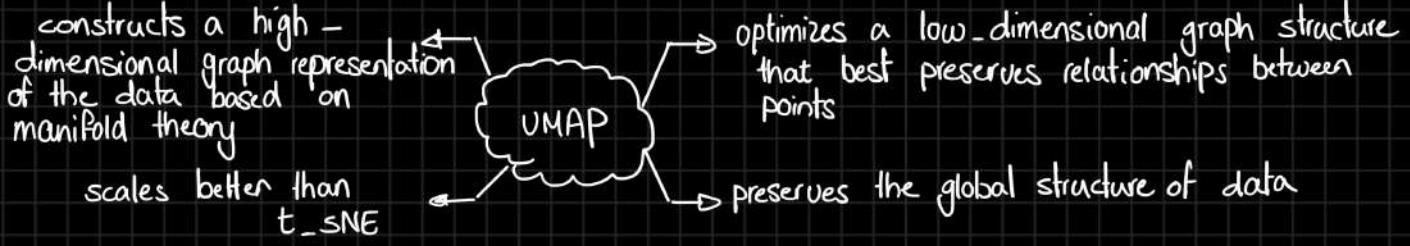
principle components => orthogonal to each other → define a new coordinate system → organized in decreasing order of importance or how much feature space variance they explain → first few components contain most of the information

* t-SNE:

Maps high-dimensional data points to a lower-dimensional space → good at finding clusters in complex, high-dimensional data → focuses on preserving similarity of points close together → similarity is measured as proximity

doesn't scale well and can be difficult to tune

* UMAP



• Classification matrix and evaluation techniques:

classification matrices:

* accuracy: ratio of correctly predicted instances to the total # of instances.

* Precision: measures how many predicted positive instances are actually +?

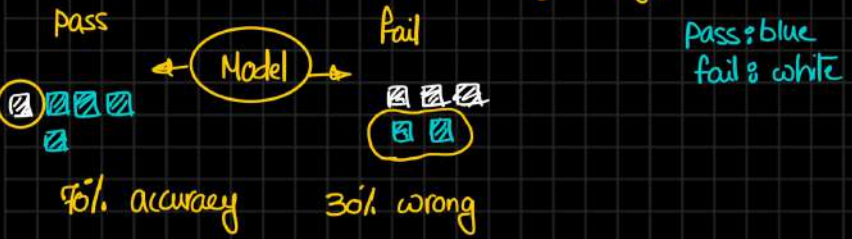
* recall: measures how many actual positive instances are correctly predicted.

* F1 score: combines precision and recall to represent a model's accuracy.

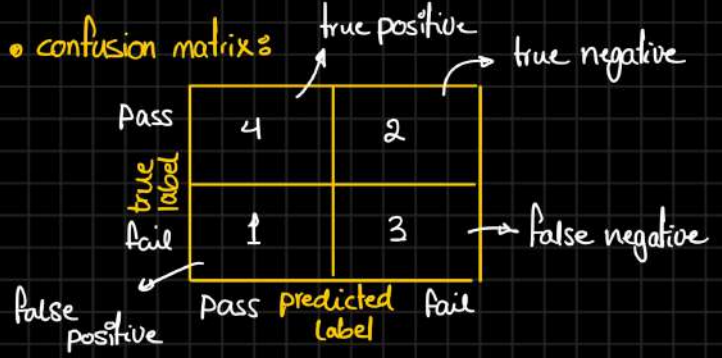
* confusion matrix: breaks down the ground truth instances of a class against the # of predicted class instances.

• accuracy:

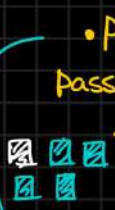
example: will I pass or fail my biology test?



• confusion matrix:



• precision → is the fraction of true positives among all the examples that were predicted to be positives.



$$\frac{\text{true positive}}{\text{true positives + false positives}}$$

where precision is more important than accuracy: recommendation engine

• recall:

$$\frac{\text{true Positives}}{\text{true positives + false negatives}}$$

→ of importance in medical industry

• F1 score:

is the harmonic mean of precision and recall

$$\frac{2 \times \text{Precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

• Regression matrix and evaluation techniques:

error → Measure of the difference between the data points and the train line generated by the algorithm.

providing insight into a model's performance

- accuracy
- error distribution
- error magnitude

MAE => Mean Absolute Error $= \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|$ (data → observed)

MSE => " squared Error $= \frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2$

RMSE $= \sqrt{MSE}$ => same units as the target variable

$$R^2 = \frac{\sum_{i=1}^{n} (\hat{y}_i - \bar{y})^2}{\sum_{i=1}^{n} (y_i - \bar{y})^2}$$ → $0 < R^2 < 1$ → perfect model; badly fit model

Negative R^2: → Model acted so poorly

unexplained variance > total variance

explained variance $= \sum_{i=1}^{n} (\hat{y}_i - \bar{y})^2$

Unexplained variance $= \sum (y_i - \hat{y}_i)^2 = n * MSE$

Total variance $= \sum (y_i - \bar{y})^2 = \sigma^2$

$$R^2 = 1 - \frac{\text{unexplained variance}}{\text{total variance}}$$

• Evaluating Unsupervised Learning Models: Heuristic and Techniques

clustering and dimensionality reduction → focus on discovering hidden patterns and structures.
assess pattern quality for model effectiveness

Unsupervised learning results → often subjective
require careful evaluation

combination of methods is essential
↓
heuristics / domain expertise / metrics / ground truth comparisons / visualization tools

important clustering heuristics
- group similar data points into clusters
- unsupervised learning lacks predefined labels
- heuristics are used to evaluate cluster quality

Heuristics
- internal evaluation matrics: rely on input data
- external evaluation matrics: use ground truth labels
- dimensionality reduction techniques: visualize clustering outcomes
- domain expertise: provides feedback
- generalizability or stability evaluation: Assesses cluster consistency
- cluster-assisted learning: refines clusters

* internal clustering evaluation metrics:

Silhouette score
↓
compares cohesion within each cluster
↓
ranges from -1 to 1
↓
higher values indicate better-defined clusters

Davies-Bouldin Index
↓
Measures cluster compactness ratio
↓
lower values indicate more distinct clusters
↓
indicates more compact clusters

Inertia
↓
used in K-means clustering
↓
calculates the sum of variances within clusters
↓
increasing clusters reduces variance

* external clustering evaluation metrics:

Adjusted rand index

Measures the similarity between true labels and outcomes

ranges from -1 to 1 for scoring

- 1 = perfect alignment
- 0 = random clustering

Negative = worse than random performance

Normalized mutual information

quantifies shared information between cluster assignments

ranges from 0 to 1

- score 1 indicates perfect alignment
- score 0 = no shared information

Fowlkes - Mallows Index

Measures clustering performance

calculates the geometric mean of precision

calculates the geometric mean of recall

higher score = better clustering results

* Dimensionality Reduction Evaluation

explained variance ratio in PCA

Measures the variance captured by principle components

helps determine the acceptable cumulative explained variance